Threshold CKKS: one key, split between R and O

Nobody ever holds $s = s_R + s_O$, so neither party can decrypt on its own

CKKS in one box

A ciphertext is a pair (c_0, c_1) of polynomials mod q. It packs a vector of N/2 = 8,192 real numbers ($N = 2^{14}$), each scaled by Ω and rounded.

Decryption with the secret key s:

$$c_0 + c_1 s \;=\; \mathrm{Ecd}(z) + e \pmod q$$

What O can do with R's ciphertexts, holding no key:

- add them
- multiply them by its own plaintext columns (x_O, y)
- rotate slots, to sum a vector into one slot (EvalSum)

Depth 2 and no ciphertext × ciphertext products, so no relinearization key. Security rests on Ring-LWE: without s, c_0 looks uniformly random.

Joint key generation (setup)

From a common random polynomial a, each party draws its own secret share and error, and publishes one polynomial:

R : $p_{0,R} = -a\, s_R + e_R$

O : $p_{0,O} = -a\, s_O + e_O$

$$pk = \big(p_{0,R} + p_{0,O},\ a\big) = \big(-a\, s + e,\ a\big)$$

Rotation keys for slot shifts 2^i are made the same way. Each share stays with its owner.

Upload under the joint key, at half the size

R : $c_0 = -a\, s_R + e + \mathrm{Ecd}(v), \qquad a = \mathrm{PRG}(\mathrm{seed})$

O : $c_0 \leftarrow c_0 - a\, s_O \quad \Rightarrow \quad c_0 + a\, s = \mathrm{Ecd}(v) + e$

R sends c_0 and the short seed of a, not a itself. O's subtraction turns (c_0, a) into a ciphertext under s.

This is standard multiparty CKKS (Mouchet et al., 2021) with two parties and additive key shares. What is specific to us is how the decryption is used: next slide.

Threshold decryption, with the DP noise built in

O adds its key share and the noise in one step; R finishes the decryption and sees only noisy statistics

Decryption needs both shares: $s = s_R + s_O$: $c_0 + c_1 s = \underbrace{(c_0 + c_1 s_O)}_{\text{needs O}} + \underbrace{c_1 s_R}_{\text{needs R}}$

Our release (phase 4)

O : $c_0' = c_0 + p_0 u + e_0, \quad c_1' = c_1 + au + e_1$

$h_O = c_0' + c_1' s_O + \eta, \quad \eta \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma_c^2)^N$

send (c_1', h_O)

R : $v = h_O + c_1' s_R = \mathrm{Ecd}(z) + e + \eta$

1. **Fresh u, e_0, e_1 every release**

   The evaluated c_1 depends on O's plaintext columns; adding $au + e_1$ makes c_1' a fresh RLWE sample that looks uniform.

2. **Add η inside O's share**

   η is the DP noise and also drowns the CKKS error e, up to 2^{-40}. It is fresh too, so two releases cannot be differenced.

3. **Never send c_0'**

   With c_0', R would get $c_1' s_O$ exactly and could solve for s_O, since c_1' is invertible.

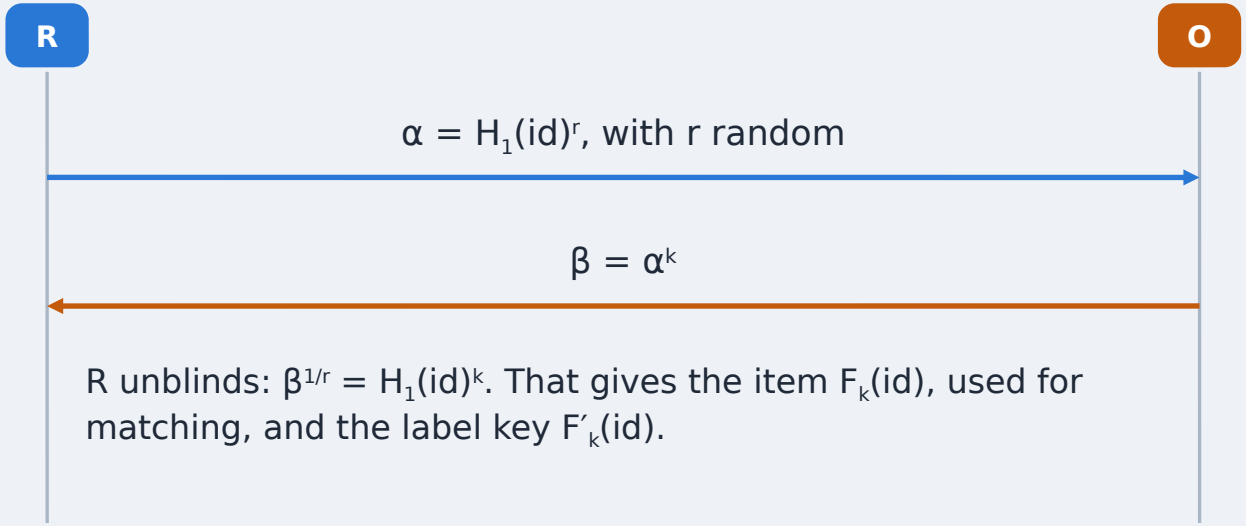
O learns nothing. It never receives a decryption share from R, so every ciphertext it holds stays hidden by Ring-LWE.

R learns the noisy statistics. Plus a c_1' that looks random, so its view can be simulated from the noisy statistics alone: the DP guarantee.

The OPRF: a keyed hash that needs O

R learns $F_k(id)$ for its own ids; O never sees the ids, and R never sees the key k

$$F_k(x) = H_2\left(x,\ H_1(x)^k\right)$$ 2HashDH (RFC 9497); O's key k is fresh each run



Why it is safe

- O sees α, a uniformly random group element: nothing about the id.
- R gets F_k only for ids it sends. Without k it cannot compute F_k for any other id.
- O answers exactly n_R^{max} queries per run (R pads with dummies) and logs them against a quota.
- A fresh k every run, so F_k values from different runs cannot be linked.

Why we need it: identifiers such as social insurance numbers are a small, guessable space, so a plain hash could be inverted by trying every value. With the OPRF every evaluation needs O, so R cannot test ids offline, and the PSI messages carry no structure. Toy example: R holds 104, 221, 317, pads with dummy 9d1 to $n_R^{max} = 4$, and makes four calls.

Labeled PSI: who matches, and where

R learns its match set I and positions π(i) = τ(j) in O's shuffled register; O learns only the padded sizes

1 O prepares

Pad the register to $\bar{n}_O$ with dummies and draw a fresh shuffle τ. Row j becomes the item $F_k(\mathrm{id}_j)$, labeled with its encrypted position. Hash the items into bins; per bin b:

$$P_b(X) = \prod_{j \in b} \left(X - F_k(\mathrm{id}_j)\right)$$
$$L_b\left(F_k(\mathrm{id}_j)\right) = \mathrm{Enc}_{F'_k(\mathrm{id}_j)}\left(\tau(j)\right)$$

2 R asks, O evaluates

R cuckoo-hashes its items, at most one per bin, encrypts them under its own BFV key, and sends their powers q, q^2, q^4, … to keep the depth low. O evaluates every P_b and L_b under encryption and replies:

$$\mathrm{Enc}_R\left(P_b(q)\right), \quad \mathrm{Enc}_R\left(L_b(q)\right)$$

3 R decrypts

$$P_b(q) = 0 \;\Rightarrow\; \text{match}$$
$$\pi(i) = \mathrm{Dec}_{F'_k(\mathrm{id}_i)}\left(L_b(q)\right) = \tau(j)$$

No match: $P_b(q) \neq 0$ and the label value means nothing. The reply reveals nothing else about O's register.

Toy example

O's padded row j	150	7e5	221	088	399	104
position τ(j)	1	2	3	4	5	6

7e5 is O's dummy row. R queries 104, 221, 317 and its dummy 9d1: 104 matches with label 6 and 221 with label 3. So I = {104, 221}, π(104) = 6 and π(221) = 3.

What each party learns

O: only $n_R^{\max}$ and $\bar{n}_O$, both public.

R: I and the positions, which are uniformly random because τ is fresh. R needs both: I for its block A, the positions to line up its upload.